ФСП Карьерный профиль

Инструкция по самостоятельному развёртыванию

11 октября 2026 года

Синтетический конкурсный стенд. Актуальные публикуемые источники и датированные протоколы сохранены в репозитории.

СОДЕРЖАНИЕ

53. Развёртывание проверяющими на своём компьютере 3
Выбор режима 3
1. Подготовка компьютера и получение кода 4
2. Проверить базовый стенд 4
3. Скачать и включить локальную Qwen 7
4. Подключить свой сервер через существующий коннектор 8
5. Эксплуатация и типовые проблемы 9
Приёмка этой инструкции 11
54. Приёмка развёртывания проверяющими · 11 октября 2026 12
Что изменилось 12
Реальные проверки 14
Состав GitHub 16
61. Временный стенд жюри на собственном домене и NAS 17
Граница стенда 17
Подготовить закрытую конфигурацию 19
Gate альтернативного прямого маршрута 19
Выбранный relay через ru2 21
Приёмка с внешней сети 24

Источник: docs/53-self-deployment.md

53. Развёртывание проверяющими на своём компьютере

Это инструкция для нового компьютера, без NAS владельца, его SSH-подключений, локальных весов или Python/Node на хосте. Код скачивается из открытого GitHub, базовая Qwen — из официального Hugging Face. Обучение для запуска не требуется: адаптеры v1/v2 отклонены, используется исходная модель без адаптера.

Выбор режима

Сведения приведены в таблице 1.

Таблица 1 — Сведения раздела «Выбор режима»

Режим	Что требуется	Что работает
Без ИИ	Docker, Compose, Git Bash/Bash	Профиль, оценки, подбор, приглашения, конструктор, задания в VM
Полный стенд с Qwen	NVIDIA GPU; модель скачивается командой ниже	Также четыре типа ИИ-черновиков и предложение оценки ответа
Свой сервер модели	Доступный OpenAI-совместимый API с JSON Schema	Те же ИИ-функции после проверки совместимости

Локальный стенд использует синтетические данные и mock ФСП; реальные данные людей вводить нельзя. Результат ИИ проверяет и подтверждает автор задания.

1. Подготовка компьютера и получение кода

Нужны Git, Docker Engine с Compose 2.24.4 или новее и Bash. Образы поставляются для Linux x86-64; Apple Silicon/AMD GPU как полный GPU-стенд этого выпуска не проверены — используйте свой совместимый сервер модели. На Windows используйте Docker Desktop в режиме Linux containers / WSL2 и Git Bash. Python, Node и PostgreSQL устанавливаются внутри образов. Для базового стенда рекомендуются 4 CPU, 8 ГБ RAM и 15 ГБ свободного диска. Для GPU-стенда рекомендуются NVIDIA с 8 ГБ VRAM, 24 ГБ RAM и 40 ГБ диска для весов, образов и кэшей; измерения выполнены на RTX 4070 Laptop 8 ГБ. 6 ГБ VRAM — нижняя граница конфигурации, а не проверенная гарантия на любой GPU.

Репозиторий открытый: для скачивания аккаунт GitHub и приглашение не нужны. Учётные данные владельца передавать не нужно. Команды для Bash:

```
git clone \
  --branch main \
  --single-branch \
  https://github.com/punkpsyh-hash/Fsp.git \
  fsp
cd fsp
docker version
docker compose version
```

В Git Bash на Windows перед дальнейшими командами:

```
export MSYS_NO_PATHCONV=1
export COMPOSE_PATH_SEPARATOR=:
```

Клонировать на компьютере владельца следует на диск D. На компьютере проверяющих выберите диск с достаточным свободным местом. Все новые веса записываются в каталог проекта; GPU-кэши Compose — в Docker volumes. Первая сборка требует доступа к Docker Hub, npm и PyPI/PyTorch; загрузка модели — к Hugging Face и его хранилищу файлов. После скачивания модель работает offline. Не запускайте одновременно разные копии с именем Compose fsp.

2. Проверить базовый стенд

```
./scripts/run.sh start
./scripts/run.sh check
```

Откройте http://127.0.0.1:8470, API — http://127.0.0.1:8470/api/docs. check показывает состояние контейнеров, проверяет API и запрос к БД. llm_provider: none означает ожидаемый режим без ИИ. На странице входа выберите синтетический аккаунт кандидата или работодателя.

Начальный экран показан на рисунке 1. На работающем стенде используется тот же интерфейс выбора демонстрационного кабинета. Данные на снимке синтетические.

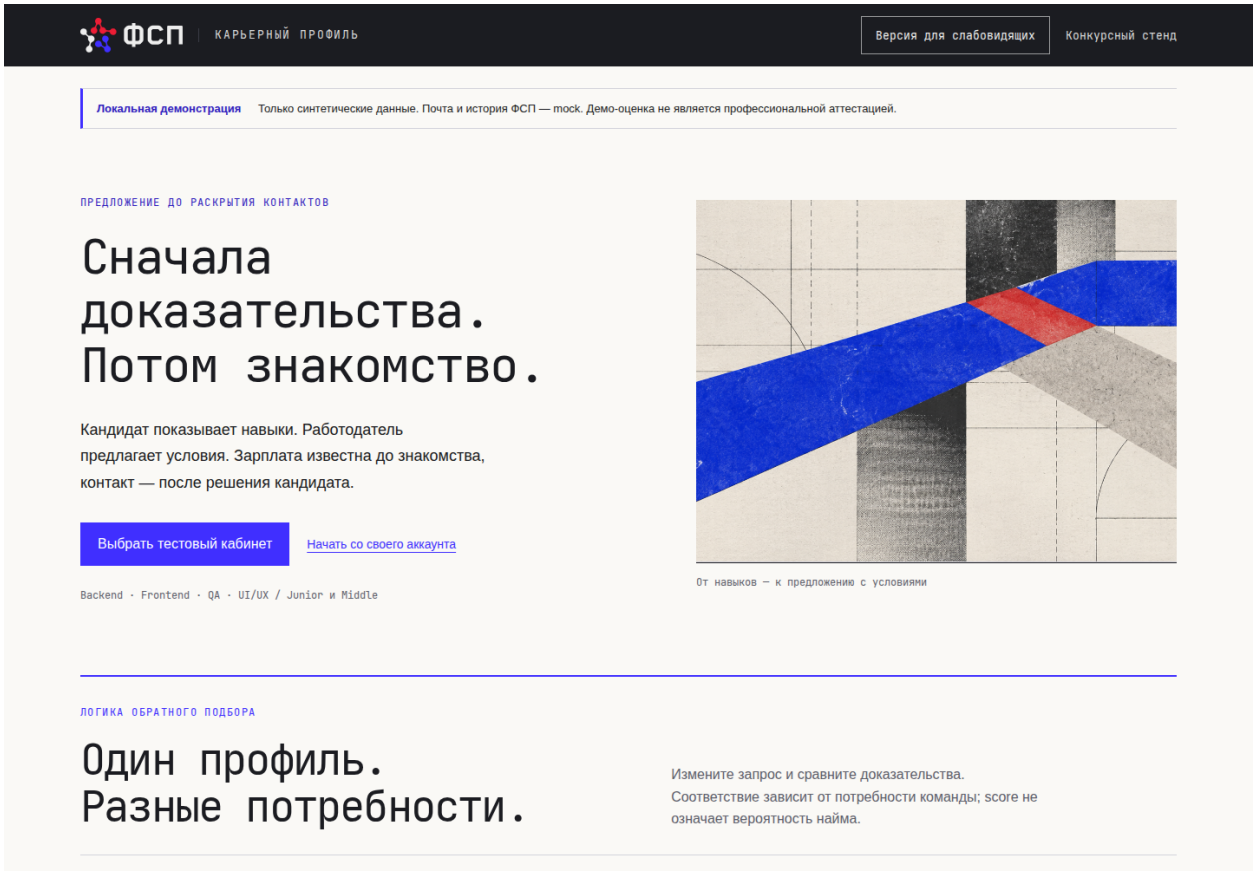


Рисунок 1 — Начальная страница и выбор демонстрационного кабинета

Источник: Работающий синтетический стенд NAS, Chromium; 11 октября 2026 года

После входа работодателя откройте «Конструктор заданий»; перечень наборов показан на рисунке 2. Проверка этого экрана не означает проверку качества ответов модели: генерация и подтверждение автором выполняются отдельными действиями.

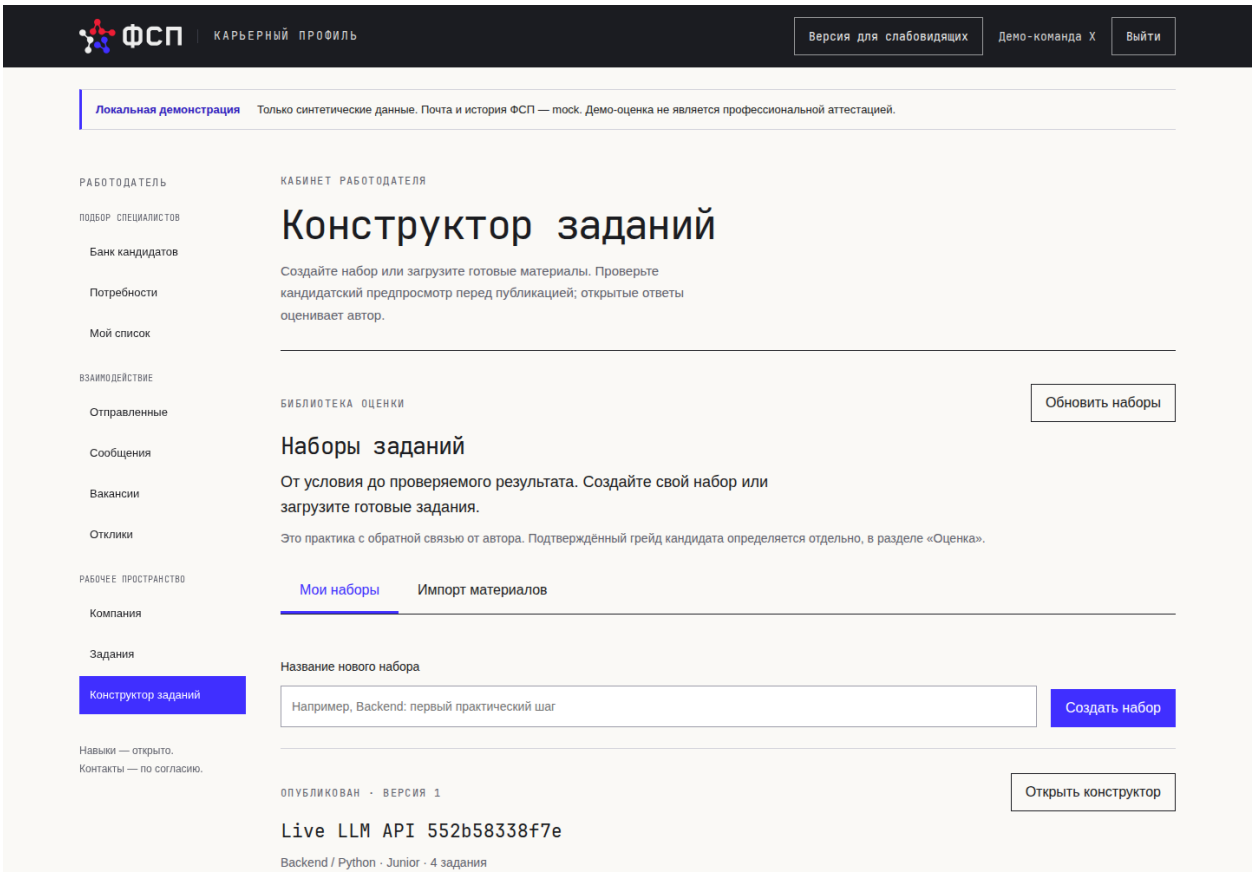


Рисунок 2 — Конструктор и версии практических наборов

Источник: Работающий синтетический стенд NAS, Chromium; 11 октября 2026 года

У работодателя проверьте подбор, банк заданий и приглашение; у кандидата — профиль, тест, добровольное практическое задание и принятие приглашения. До согласия кандидата контакты работодателю недоступны.

Скрипт создаёт .env со случайными секретами, сохраняет прежние значения, применяет миграции и идемпотентный seed. Не копируйте .env.example поверх созданного .env. Если /dev/kvm недоступен, автоматически используется медленный TCG; пользовательский код всё равно выполняется в отдельной VM.

3. Скачать и включить локальную Qwen

Для Linux нужен настроенный NVIDIA Container Toolkit. Для Docker Desktop GPU должен быть доступен Linux-контейнерам через WSL2. Проверьте это до многочасовой сборки:

```
docker run --rm --gpus all ubuntu:24.04 nvidia-smi
./scripts/run.sh model-fetch
./scripts/run.sh model-verify
./scripts/run.sh start-llm
```

Загрузчик получает около 8 ГБ весов из официального репозитория Qwen. Версия закреплена полным коммитом cdbee75f17c01a7cc42f958dc650907174af0554; SHA-256 каждого файла — в ml/release/model-serving-v3-manifest.json. GitHub содержит загрузчик и manifest, а не многогигабайтные веса. Модель опубликована авторами под Apache-2.0; её LICENSE загружается вместе с весами. Это не выбирает лицензию самого проекта.

При обрыве повторите model-fetch: проверенные файлы будут пропущены, незавершённый файл скачивается заново. model-verify сверяет хеши offline, не загружает модель в GPU и завершится с ошибкой при пропуске, изменении или лишнем файле. Каталог весов — ml/models/qwen3-4b-instruct-2507-serving-v3/. Не кладите туда свои заметки, адаптеры или другой tokenizer.

start-llm сначала проверяет файлы, собирает образы и ждёт готовности. Ключ локальной модели создаётся автоматически в .env; модель и приложение получают одинаковый ключ. Сервис выполняет обязательный warmup до healthy. Модель не публикует порт на хост; сеть llm_net внутренняя, веса read-only. Прогрев проверенного образа занял около 203 секунд; первый download/build может занять значительно дольше и не входит в эти 203 секунды.

Для последующих команд в каждом новом Bash-терминале выберите тот же набор:

```
export COMPOSE_FILE=compose.yaml:compose.llm.yaml
./scripts/run.sh check-llm
./scripts/run.sh status-llm
```

На Windows скрипт автоматически добавит TCG overlay. Ожидается healthy у db, app, runner, model, а в выводе check — local_model: ready. Проверка подтверждает модель, её версию и ключ, но не правильность

нового задания. Для проверки генерации войдите работодателем, откройте «Банки заданий» / конструктор, запросите по одному заданию single, multiple, numeric, open. Автор проверяет условие, эталон, варианты и рубрику перед сохранением и публикацией. Подсказка ИИ по ответу тоже требует ручного решения; практическое задание не повышает глобальный грейд. Проверенный путь и ограничения — приёмка.

4. Подключить свой сервер через существующий коннектор

Вместо compose.llm.yaml можно использовать реализованный OpenAICompatible в src/server/llm.py: это административная настройка, а не поле URL, доступное пользователю. Поддерживается POST <base_url>/chat/completions с Bearer-ключом, messages, max_tokens, response_format.type=json_schema, строгой схемой и finish_reason=stop. Обычного текстового chat API для генератора недостаточно.

После ./scripts/run.sh init отредактируйте только указанные строки .env, сохранив сгенерированные FSP_DB_PASSWORD, FSP_RUNNER_TOKEN и остальные настройки:

```
FSP_LLM_PROVIDER=openai-compatible
FSP_LLM_BASE_URL=https://your-model-server.example/v1
FSP_LLM_MODEL=your-model-id
FSP_LLM_API_KEY=your-own-local-key
FSP_LLM_TIMEOUT=100
FSP_LLM_GENERATION_TIMEOUT=120
```

Значения выше — пример, адрес .example не является рабочим сервером. Укажите адрес, доступный из контейнера app. Внешний сервер требует HTTPS; внутренний локальный сервис допускает HTTP. localhost внутри app означает сам контейнер, а не компьютер с моделью. Для Docker Desktop используйте host.docker.internal, если модель слушает доступный контейнерам адрес. Обращение к уже работающему локальному серверу не устанавливает ему модель. При сетевом доступе ключ и настройки TLS задаются администратором.

```
unset COMPOSE_FILE
./scripts/run.sh start
./scripts/run.sh check
```

В этом режиме check проверяет конфигурацию и базовый стенд; он не делает платный запрос и не объявляет произвольную модель совместимой.

Совместимость проверяется четырьмя заданиями в конструкторе и подсказкой оценки. Собственная Qwen на GPU проверена; произвольные внешние провайдеры отдельно не сертифицированы. Не передавайте реальные персональные данные в модель. Если ранее включали Compose-модель, сначала остановите её по пункту 5, затем переключите конфигурацию; unset COMPOSE_FILE само не останавливает контейнер.

5. Эксплуатация и типовые проблемы

Для GPU-режима сохраняйте COMPOSE_FILE=compose.yaml:compose.llm.yaml:

```
./scripts/run.sh logs
./scripts/run.sh model-report
./scripts/run.sh backup
./scripts/run.sh stop-llm
./scripts/run.sh start-llm
```

stop-llm останавливает выбранный проект, включая модель, и сохраняет БД и кэши. backup сохраняет dump в игнорируемый каталог backups/; инструкции по восстановлению — эксплуатация. Не используйте down -v или глобальный Docker prune для обычного перезапуска.

Сведения приведены в таблице 2.

Таблица 2 — Сведения раздела «5. Эксплуатация и типовые проблемы»

Симптом	**Что проверить**
GitHub 404 / authentication failed	Верно ли указан публичный адрес https://github.com/punkpsyh-hash/Fsp; исправны ли DNS и соединение с GitHub
Compose не понимает !override / !reset	Обновить Compose до 2.24.4+
Порт 8470/15432 занят	Остановить свою прежнюю копию; не останавливать чужие сервисы
GPU недоступна	nvidia-smi, Container Toolkit / WSL2, режим Linux containers
SHA-256 / недостающий файл	Повторить model-fetch, затем model-verify; не менять manifest
Model unhealthy / CUDA OOM	Освободить свою GPU-нагрузку, проверить VRAM; не включать CPU-offload обходом
check отклоняет ключ	Сверить конфигурацию .env, пересоздать model/app штатным start-llm
Задание не сгенерировано	Посмотреть машинную причину в UI; timeout/ошибка схемы не означают публикацию
Медленное практическое задание	Проверить KVM; TCG сохраняет VM-границу, но работает медленнее
Docker Desktop не стартует	Диагностика Docker Desktop

Дополнительная регрессия: ./scripts/run.sh test, test-postgres, e2e, runner-test, contest-bank. Эти команды могут занять десятки минут. Протокол именно этой доработки публикуется отдельно от исторической приёмки.

Если Docker сообщает all predefined address pools have been fully subnetted, его автоматические диапазоны исчерпаны другими сетями. Не удаляйте чужие сети. Администратор может задать свободные, не пересекающиеся с LAN/VPN/Docker подсети в локальном Compose overlay; проверенный пример использует этот способ. Диапазоны в примере нельзя переносить без проверки сети конкретного компьютера.

Приёмка этой инструкции

Для временного показа на собственном домене используйте отдельную инструкцию NAS и jury-контура. Это парольное демо на синтетических аккаунтах с отключёнными регистрацией, письмами и импортом резюме; базовый локальный режим сохраняет полный регистрационный путь. Публичный запуск требует действительных сведений владельца и email в политике, DNS, HTTPS и отдельной внешней приёмки. Название «тестовый стенд» не заменяет эти условия. Внешняя почта по умолчанию выключена; одного FSP_SMTP_HOST недостаточно: нужно явное FSP_SMTP_ENABLED=1, TLS и проверка условий поставщика по docs/58.

Проверяем: получение кода без локальных весов/секретов; создание .env и повтор без перезаписи; запуск БД/API/runner из чистого дерева; download и offline SHA-256; отказ при повреждении; состав Compose с GPU и без GPU; работа коннектора на контрактных тестах; отсутствие закрытых данных и CI/CD в публикуемом дереве. Результаты и точный срез — docs/54.

Источник: docs/54-self-deployment-acceptance.md

54. Приёмка развёртывания проверяющими · 11 октября 2026

Доработка выполнена поверх f786ae85edcfb4ba09c898028c519f023d019cfc. Серверные правила, интерфейс, модели данных и VM-граница не изменялись. Протокол этого этапа — docs/release/2026-10-11-self-deployment/verification.json. Предыдущие полные 410/411 тестов и 132 браузерных сценария относятся к предыдущему принятому срезу, а не к новому полному прогону.

Что изменилось

- Подготовлены краткий README и навигация по актуальной документации. Предыдущие редакции доступны в Git history.
- Добавлена самостоятельная инструкция docs/53: базовый стенд, загрузка Qwen, GPU-запуск и собственный API модели.
- run.sh преобразует Windows-пути через cygpath, задаёт параметры Git Bash, поддерживает model-fetch, model-verify, start-llm, status-llm, check-llm, stop-llm, model-report. Установка Python/Node на хост не нужна.
- init_env.py создаёт случайный ключ локальной модели без перезаписи остальных секретов. Пустой существующий ключ отклоняется без изменения .env.
- GPU Compose требует ключ; генерация ограничена 120 секундами, обычный запрос модели — 100. Первый startup ждёт до 900 секунд; PostgreSQL получает start period 120 секунд. Бюджеты CPU/RAM самой модели не повышались.
- Download использует закреплённый commit и SHA-256, временный .part и замену только после проверки. Проверка offline отклоняет пропуски, изменения, лишние файлы, ссылки и небезопасные пути.
- ИИ-коннектор остаётся существующим OpenAICompatible; новая абстракция или внешние учётные данные не добавлены.
- Transfer metadata, кэши и GitHub workflows исключены из Docker-контекста; локальный TRANSFER-MANIFEST.json сохранён и исключён из Git.

Исторические генераторы, протоколы, презентации и видео не удалены: на них ссылаются проверки и документация. «Очистка» означает понятные входные документы, отделённую историю и исключение частных/временных файлов из передачи.

Реальные проверки

Сведения приведены в таблице 3.

Таблица 3 — Сведения раздела «Реальные проверки»

Проверка	Результат и граница
Docker-сборка из чистого локального клона	Собраны runtime frontend/backend и VM runner; локальные веса/секреты не использовались
Чистая БД/API/runner	Все три контейнера healthy; /api/health подтверждает запрос к PostgreSQL
Повтор init	SHA-256 .env до/после одинаков; секреты в отчёт не включены
Базовый check	API/БД ready, провайдер none
check-llm на базовом app	Отказ с exit 1; доступный сайт не выдаётся за готовность модели
TCG smoke отдельного runner	6/6: реальная VM, Python, JavaScript, SQLite, CPU timeout, неверный ответ
Целевая регрессия	69 тестов: download, local model, LLM, Unix transport, конфигурация, startup и ready-check
Реальная работа HTTPS-download	9 файлов официального pinned Qwen получены из Hugging Face и сверены; три больших shard из сети заново не скачивались
Offline полных весов	Все 12 файлов сверены по manifest; при повреждённом shard команда предварительно вернула exit 1
Ready-check настоящей GPU-модели	Подтверждены точный Qwen/revision, NF4 CUDA:0 и ключ; неправильный ключ отклонён. API health в этой адресной пробе был stub, отдельно реальный API проверен на чистом стенде
Инициализация конфигурации	Проверены сохранение прежних байтов, создание ключа и идемпотентность, отказ на blank и symlink
GPU Compose	Проверена сборка конфигурации с ключом и без KVM; пустой ключ отклоняется

Новый холодный запуск полного GPU Compose здесь не повторялся, чтобы не останавливать существующий GPU/NAS-стенд. Готовность

реального текущего GPU и его авторизация проверены; живые четыре типа генерации и NAS-цикл остаются доказательством предыдущей приёмки. Сторонний внешний провайдер и самостоятельный запуск на компьютере жюри не проверены. Unit-тесты коннектора не выдаются за сертификацию любого API.

Восстановление локального shard

Первая offline-проверка обнаружила изменённый model-00001-of-00003.safetensors: SHA-256 c98e31b30e994ecdf4e020eacd5a16bea2c1e9ca08befdc8173e7efad1c5b1f5. Предыдущие байты сохранены отдельно на D вне Git. Имеющаяся исходная копия совпала с manifest (75311d91bb08cf0b882913da464a1e722a31fb44db35208663487efb7a3d8ed6) и заменена после проверки. Происхождение повреждения не установлено. Manifest не изменялся; работающий сервис не останавливался.

Ограничения тестового окружения

На компьютере владельца автоматические Docker address pools исчерпаны другими проектами. Проверялись существующие подсети; для отдельного проекта fspselfdeploy выбран не пересекающийся локальный диапазон 10.253.238.0/24. Host-порты отключены, образы и volumes изолированы. Это отличие лабораторной пробы от штатного запуска на свободном компьютере описано явно. В локальном local-data/selftest.yaml использовалось дополнение:

```
services:
  db:
    ports: !reset []
  app:
    image: fspselfdeploy-app
    ports: !reset []
  runner:
    image: fspdev-runner
networks:
  default:
    ipam:
      config: [{subnet: 10.253.238.0/24}]
```

Это пример изолированного теста, а не универсальный адрес для чужой сети. Существующий runner image был пересобран из чистого клона; имя

image не является именем Compose-проекта. Владелец не удалял чужие сети и данные. Штатный пользовательский запуск использует ports из compose.yaml.

Состав GitHub

Публикуются исходники, контейнерные рецепты, инструкции, тесты, manifest, анонимизированные требования, документы и готовый PPTX. Не публикуются .env, ключи, веса, адаптеры, БД, backup, local-data, новые оригиналы брендбука/шаблона, оригинальный XLSX и .github/workflows/. На исходном этапе изменения отправлялись в закрытую ветку codex/local-llm-workspace и PR 1. Этот порядок заменён решением владельца 11 октября: очищенный конкурсный выпуск публикуется как Fsp/main, старая история сохраняется в закрытом Fsp-private-history. Проверяющим не нужно приглашение GitHub; актуальные ссылки — docs/60.

Источник: docs/61-temporary-domain-stand.md

61. Временный стенд жюри на собственном домене и NAS

Стенд https://fsptest.kolesnikovny.ru опубликован 11 октября 2026 года через российский VPS ru2 и зашифрованный обратный SSH-туннель к NAS. Отдельный Compose-проект fspjury работает в каталоге /Volume1/projects/Fsp-jury-20261011. Публичный IP — 80.87.198.60; сертификат Let's Encrypt проверен внешним клиентом и действует до 9 января 2027 года. Внешний HTTP-протокол прошёл 22 проверки; VM-runner — 28 проверок с KVM. Протоколы — jury-public-http.json и jury-vm.json в docs/release/2026-10-11-legal-submission/.

Первоначальная диагностика обнаружила старую A-запись 5.101.153.251, исходящий адрес NAS 46.8.231.30 и частный WAN шлюза 10.100.237.64. Поэтому прямой проброс текущего роутера не использован. Владелец подтвердил размещение NAS в России и поручил выбрать российский VPS; это сведения владельца, а не независимое доказательство страны по IP. Административные порты NAS не опубликованы. Пароли и выделенный private key остаются вне Git.

ТЗ §3.3 и §5.2 крайне рекомендует ссылку на развёрнутый прототип, а при её отсутствии разрешает видео и воспроизводимую локальную инструкцию. Письменный ответ E10/B3 разрешает видео и рекомендует держать стенд весь период оценки. Поэтому этот контур дополняет уже готовые видео и docs/53-self-deployment.md, а не заменяет их и не создаёт обещание непрерывной доступности.

Для этого NAS выбран relay через российский VPS ru2: раздел «Выбранный relay через ru2» и команды scripts/jury-relay-nas.sh. Прямой маршрут с пробросом роутера ниже приведён как альтернатива для сети с подтверждённым публичным WAN; на текущем NAS он не используется.

Граница стенда

- Compose-проект всегда называется fspjury. Он создаёт отдельные образы, PostgreSQL volume и volumes Caddy; прежние проекты fsp/fspdelivery и работающий fspdelivery-app-1 не используются и не останавливаются.

- PostgreSQL и app не публикуют host-порты. Для выбранного relay внешняя точка — HTTPS nginx ru2; NAS Caddy доступен только на loopback TCP 18083. LAN-порты 18083/18443 ниже относятся только к альтернативному прямому маршруту.
- Системный nginx NAS на 80/443, административные 8181/5443 и его конфигурация не изменяются. На роутере допустимы только правила WAN TCP 80 → 192.168.1.58:18083 и WAN TCP 443 → 192.168.1.58:18443.
- Caddy 2.11.7 взят из официального образа и закреплён index digest sha256:d8542f48d34a9cf4e4c11a478865229840e87e4c96ea3f439101f31a5d35f75f. Сертификат получает Caddy напрямую по ACME; CDN и чужой TLS-терминатор не нужны.
- Access log Caddy не включён, чтобы не собирать IP посетителей без цели. Короткие runtime-логи нужны только для диагностики сертификата и состояния.
- На отдельной jury-сети Caddy получает фиксированный адрес 10.254.242.10. Uvicorn доверяет X-Forwarded-For и forwarded scheme только от этого адреса: прямой контейнер или клиент не может подменить свой адрес таким заголовком. Если подсеть меняют через FSP_JURY_SUBNET, одновременно задают принадлежащий ей свободный FSP_JURY_EDGE_IP; доверие ко всей подсети или * запрещено.
- /internal/* снаружи всегда отвечает 404. /legal/privacy и /legal/recommendations открыты до входа, чтобы показать фактического оператора и правила обработки. Все остальные пути сначала требуют отдельный Basic Auth, затем пароль готового синтетического аккаунта приложения.
- В jury-режиме регистрация, вход по ссылке/коду, mock-почта, ФСП ID mock, загрузка резюме и внешний SMTP выключены. Используются только синтетические seed-данные; реальные данные вводить нельзя. Приложение запускается с FSP_JURY_MODE=1, secure-cookie и allowlist одного HTTPS origin.

Подготовить закрытую конфигурацию

Команды выполняются в отдельной рабочей копии проекта на NAS. Они не меняют роутер, DNS или системный nginx.

```
chmod +x scripts/jury.sh
./scripts/jury.sh init
./scripts/jury.sh config
./scripts/jury.sh validate
```

init интерактивно запрашивает фактическое имя и действительный публичный email оператора, показывает их повторно и требует явное подтверждение OPERATOR-CONFIRMED. Четыре независимых секрета создаются через secrets.token_urlsafe(48): пароль приложения, пароль Basic Auth, пароль БД и runner token. Caddy формирует Argon2id hash локально. Пароли и hash записываются только в игнорируемый Git .env.jury с umask 077 и не выводятся скриптом. Файл не перезаписывается автоматически. Администратор читает пароли только локально из .env.jury, передаёт два пользовательских пароля проверяющим отдельным защищённым каналом и меняет их после окончания оценки.

Для этого стенда подтверждены несекретные реквизиты: оператор данных — Колесников Никита Юрьевич, публичный контакт — inbox@nkolesnikov.ru, страна размещения и хранения — RU. Именно эти имя и email нужно ввести в init и проверить на страницах политики. Пароли и hash в документацию не переносят.

./scripts/jury.sh config проверяет env и Compose, затем показывает только несекретные параметры. Нельзя запускать или сохранять полный вывод docker compose config: в развёрнутой конфигурации находятся пароль БД, runner token, пароль приложения и hash Basic Auth. Для автоматической проверки используется только docker compose config --quiet, без tee и перенаправления.

Gate альтернативного прямого маршрута

Владелец сети должен фактически подтвердить каждый пункт:

1. Выбран и отдельно проверен входящий маршрут. Текущий шлюз сообщает частный WAN 10.100.237.64, поэтому исходящий адрес 46.8.231.30 нельзя помещать в DNS как адрес NAS без подтверждения провайдера. Для прямого варианта нужен выделенный глобальный IPv4

на текущем роутере либо управляемый внешний роутер с публичным адресом и явным пробросом до него. Адрес должен сохраняться на период оценки либо DNS должен своевременно обновляться.

2. В панели DNS A для fsptest.kolesnikovny.ru указывает на проверенную точку входа выбранного маршрута; лишней AAAA-записи, ведущей на недоступный IPv6, нет. NS и доступ к панели проверены.
3. NAS по-прежнему имеет зарезервированный DHCP/LAN 192.168.1.58; TCP 18083 и 18443 свободны. Никакие правила не направлены на 80/443 самого NAS.
4. Роутер пробрасывает только WAN TCP 80 → 192.168.1.58:18083 и WAN TCP 443 → 192.168.1.58:18443. UPnP для проекта не используется. UDP 443 не требуется: HTTP/3 можно не публиковать.
5. Фаервол разрешает эти два TCP-направления. Проверка выполняется с другой сети, например мобильной, а не только из домашнего LAN.
6. Фактические имя/email оператора опубликованы на страницах политики и администратор зафиксировал FSP_JURY_OPERATOR_CONFIRMED=1 после проверки. Подтверждённое владельцем размещение NAS в России отражено как FSP_JURY_STORAGE_COUNTRY=RU; это подтверждение владельца, а не вывод по GeoIP. Установлен срок удаления jury-volume после оценки.
7. В app включён jury-режим: /api/health сообщает mode: jury-demo, вход работает только паролем готовых аккаунтов, опасные публичные методы дают 404. В репозитории нет .env.jury и самих паролей; полную развёрнутую Compose-конфигурацию не выводят, потому что она содержит секреты.
8. Проверена граница reverse proxy: два клиента через Caddy получают разные ключи ограничения входа, а прямой запрос к app с поддельным X-Forwarded-For не меняет фактический адрес клиента.

Для альтернативного прямого маршрута после подтверждения публичного WAN и проброса внесите подтверждённый адрес в FSP_JURY_PUBLIC_IP, задайте в .env.jury ровно FSP_JURY_ACTIVATE=DNS_AND_ROUTER_VERIFIED и выполните:

```
./scripts/jury.sh gate
./scripts/jury.sh start
./scripts/jury.sh status
```

gate сверяет A-запись с указанным IPv4 и Compose-конфигурацию. Он не может доказать проброс портов или статичность адреса, поэтому внешняя проверка нужна отдельно. Caddy требует доступ CA к WAN 80 или 443; первый выпуск сертификата может не состояться до корректного DNS и проброса.

Выбранный relay через ru2

Из двух имеющихся серверов выбран менее загруженный ru2: IPv4 80.87.198.60, hostname inbox1.fvds.ru, Ubuntu 24.04, свободно около 2,6 GiB RAM. ru1 не используется, потому что на нём уже работают несколько CRM. Эти сведения нужны для выбора узла, но сами по себе не доказывают страну физического размещения. Российский VPS выбран по поручению владельца; для промышленной эксплуатации сведения о физическом размещении подтверждают договором провайдера. VPS будет обрабатывать IP посетителей как технический reverse proxy; страницы политики должны описывать этот маршрут.

Публичный путь:

```
браузер → HTTPS nginx ru2 → 127.0.0.1:18470 ru2
        → зашифрованный reverse SSH → 127.0.0.1:18083 NAS
        → Caddy relay → app:8470
```

- Существующий nginx на VPS сохраняет глобальную конфигурацию и остальные vhost. Добавляется только exact vhost fsptest.kolesnikovny.ru из docker/jury/nginx-vps.conf; он проксирует только на loopback 18470.
- nginx перезаписывает полученный от клиента X-Forwarded-For своим $remote_addr и задаёт X-Forwarded-Proto https. Поэтому посетитель не может подменить адрес входным заголовком.
- На VPS отдельный SSH user fspjury-tunnel разрешает только remote forwarding и только listener 127.0.0.1:18470. В authorized_keys одновременно нужны restrict, явное port-forwarding и permitlisten="127.0.0.1:18470".
- На NAS Caddy публикуется только на 127.0.0.1:18083, работает по HTTP и доверяет forwarded headers только bridge gateway 10.254.242.1. Перед app он заново задаёт X-Forwarded-For {client_ip} и X-Forwarded-Proto https.

- Relay gate фиксирует default subnet 10.254.242.0/24 и адрес Caddy 10.254.242.10; менять их без одновременного пересмотра trust boundary нельзя.
- Uvicorn по-прежнему доверяет только Caddy 10.254.242.10. App сохраняет штатные default и изолированную runner_net; runner запускается с прежней VM-границей и бюджетами, чтобы задания по программированию оставались доступны.
- TLS завершается на VPS, а участок VPS–NAS шифрует SSH. При падении туннеля nginx получает ошибку loopback upstream и отвечает 502; обхода напрямую к app или Caddy по LAN/WAN нет.

Relay развёрнут назначенным оркестратором. Скрипты ниже описывают воспроизводимую подготовку и имеют отдельные проверки и изменяющие команды.

Подготовка NAS без запуска

./scripts/jury.sh init создаёт .env.jury как описано выше. В этот локальный игнорируемый файл добавляют только выбранный узел и пути к ключам вне Git:

```
FSP_JURY_VPS_HOST='80.87.198.60'
FSP_JURY_VPS_SSH_PORT='22'
FSP_JURY_TUNNEL_USER='fspjury-tunnel'
FSP_JURY_SSH_KEY_PATH='/Volume1/projects/Fsp-local/jury-relay/
  id_ed25519'
FSP_JURY_SSH_KNOWN_HOSTS_PATH='/Volume1/projects/Fsp-local/jury-
  relay/known_hosts'
FSP_JURY_RELAY_ACTIVATE=''
```

Создаётся отдельная Ed25519-пара только для туннеля. Private key остаётся на NAS вне Git с режимом 0600 и владельцем UID 10001 контейнера; на VPS передаётся только .pub. known_hosts заполняют для 80.87.198.60, а fingerprint Ed25519 сверяют через уже доверенный канал управления ru2; одного ssh-keyscan без сверки недостаточно.

```
./scripts/jury-relay-nas.sh config
./scripts/jury-relay-nas.sh validate
```

Точечная подготовка VPS

scripts/jury-relay-vps.sh закреплён за 80.87.198.60. preflight только читает состояние и отказывает, если домен уже объявлен другим vhost. Следующие команды изменяют VPS и выполняются оркестратором по этапам:

```
./scripts/jury-relay-vps.sh preflight
./scripts/jury-relay-vps.sh install-ssh /secure/path/
  id_ed25519.pub
./scripts/jury-relay-vps.sh install-http
# После DNS A → 80.87.198.60 и внешней проверки HTTP challenge:
./scripts/jury-relay-vps.sh issue-cert
./scripts/jury-relay-vps.sh install-https
./scripts/jury-relay-vps.sh status
```

Для этого vhost access log выключен, а отдельный error log ограничен собственным правилом logrotate: ротация при 1 MiB, семь архивов. Общие настройки и журналы nginx не изменяются.

Для выпуска используется официальный Certbot 5.8.0, закреплённый index digest sha256:f70ad0adbb7e117f0fe42a63c553f28ea451edabc0148757b6efcd9735acaa20, с webroot только этого vhost. Существующий ACME account переиспользуется, а при его отсутствии создаётся account без email: публичный адрес оператора не меняет назначение на рекламные или CA-уведомления. renew повторяет webroot renewal и перезагружает nginx только после nginx -t. Расписание renewal на VPS проверяется отдельно. На выбранном ru2 уже включён certbot.timer с Certbot 5.6.0 и reload nginx; его настройки сохранены. Webroot в контейнере и на VPS одинаков: /var/www/fsptest-acme. Успешный reconfigure проверил staging-продление и сохранил этот путь; ручная команда renew ограничена только сертификатом данного домена.

Активация relay

До изменения DNS приватный путь поднимают отдельной командой; edge NAS при этом доступен только на loopback, а HTTP bootstrap VPS отвечает 503:

```
./scripts/jury-relay-nas.sh prepare
```

После этого оркестратор проверяет на VPS listener 127.0.0.1:18470, health через туннель и работающий runner. Только после подтверждения страны VPS, SSH fingerprint, DNS, сертификата, loopback listener и внешних тестов задают ровно

FSP_JURY_RELAY_ACTIVATE=VPS_TLS_AND_TUNNEL_VERIFIED, затем:

```
./scripts/jury-relay-nas.sh gate
./scripts/jury-relay-nas.sh start
./scripts/jury-relay-nas.sh status
```

Из внешней сети повторяют все проверки раздела «Приёмка», отдельно проверяют два разных client IP в rate limit, поддельный входной X-Forwarded-For, 404 для /internal и fail-closed 502 при контролируемой остановке только tunnel. Публичный запуск до этих проверок не заявляется. Если relay не будет готов к сроку, резерв остаётся прежним: видео и воспроизводимая локальная инструкция.

Приёмка с внешней сети

Не помещайте пароли в историю shell или диагностические файлы. Введите их в браузере и проверьте DevTools либо используйте интерактивные переменные shell.

```
read -r -p 'Basic user: ' JURY_USER
read -r -s -p 'Basic password: ' JURY_EDGE_PASSWORD; printf '\n'
curl -fsS https://fsptest.kolesnikovny.ru/legal/privacy >/dev/
  null
curl -sS -o /dev/null -w '%{http_code}\n' https://
  fsptest.kolesnikovny.ru/
curl -fsS -u "$JURY_USER:$JURY_EDGE_PASSWORD" https://
  fsptest.kolesnikovny.ru/api/health
curl -sS -u "$JURY_USER:$JURY_EDGE_PASSWORD" -o /dev/null -w
  '%{http_code}\n' https://fsptest.kolesnikovny.ru/internal/
  runner/claim
unset JURY_EDGE_PASSWORD
```

Ожидается: legal policy — 200 без Basic Auth; / без Basic Auth — 401; health с Basic Auth — 200 и jury-demo; /internal/... — 404 даже с паролем. Далее в браузере проверить password-only вход кандидата и работодателя, отсутствие регистрации/почты/ФСП ID/загрузки резюме, secure session cookie, изменяющий запрос с корректным CSRF и такой же запрос без CSRF (403). Проверить основной сквозной сценарий на синтетических данных и выход из сессии.

Не заявляйте стенд доступным, пока HTTPS и эти проверки не пройдены из внешней сети. Для выбранного relay на период оценки контролируйте ./scripts/jury-relay-nas.sh status и ограниченный

./scripts/jury-relay-nas.sh logs; не включайте access log. Остановка сохраняет данные:

```
./scripts/jury-relay-nas.sh stop
```

После согласованного срока хранения удаление только отдельного jury-контура выполняется явно: ./scripts/jury-relay-nas.sh reset --yes. Команда удаляет контейнеры, сети и volumes проекта fspjury; .env.jury удаляется вручную после проверки, что он больше не нужен. Прежние NAS-проекты команда не затрагивает.

Источники настройки Caddy: Automatic HTTPS, Basic Auth и официальный Docker image.